CECS 447 Final Project

I2C Bus Communication System

Designed by Jackie Huynh and Min He · April 18, 2026

Course: CECS 447 (Senior Embedded Systems) · Duration: April 20 — May 13, 2026

5 Module Specifications

Note on Starter Code

Each module is provided as a .c/.h file pair with CONSTANT_FILL and CODE_FILL placeholders. Function names, data types, and struct names below reflect the actual starter code signatures. Do not rename functions or change their signatures.

5.1 Module 1: Onboard LED and Button Control (ButtonLED.c / ButtonLED.h)

Property	Value
Purpose	Initialize onboard RGB LED (PF1/PF2/PF3) and push buttons SW1 (PF4) / SW2 (PF0)
Hardware	Port F: LED on PF[3:1], SW1 on PF4, SW2 on PF0
Dependencies	tm4c123gh6pm.h

Functional Requirements

Function Signature	Description
`void LED_Init(void)`	Initialize Port F GPIO for LED output on PF[3:1]
`void BTN_Init(void)`	Initialize Port F GPIO for button input on PF4 and PF0; configure pull-up resistors for SW1 and SW2

Expected Behavior (integrated with WTIMER unit test)

- Red LED blinks at 2 Hz using DELAY_1MS(500) in a loop
- SW1 press toggles between UART test mode and WTIMER delay mode
- SW2 press (in timer mode) cycles LED color: RED → GREEN → BLUE → RED …

Test Criteria

- LED_Init and BTN_Init complete without system fault
- All three LED colors (RED, GREEN, BLUE) are individually illuminated on demand
- SW1 press changes system mode; SW2 press cycles LED color

5.2 Module 2: Wide Timer (wtimer.c / wtimer.h)

Property	Value
Purpose	Provide 1 millisecond delay function used by all other modules; general-purpose value mapping
Hardware	32-bit wide timer peripheral (WTIMER1–5, student choice; WTIMER0 not allowed)
Clock	System clock with prescaler for 1 ms resolution
Header guard	UTIL_H_ (wtimer.h uses the #ifndef UTIL_H_ guard)
Dependencies	tm4c123gh6pm.h

Functional Requirements

Students must implement the following functions, which are declared in wtimer.h and stubbed in wtimer.c:

Function Signature	Description
`void WTIMER_Init(void)`	Initialize wide timer peripheral for 1 ms delay operation
`void DELAY_1MS(uint32_t delay)`	Block execution for the specified number of milliseconds

CONSTANT_FILL Macros (wtimer.h)

The following macros must be replaced with correct register values:

```
#define EN_WTIMER_CLOCK    (CONSTANT_FILL)  // SYSCTL clock-gate bit for wide timer
#define WTIMER_TAEN_BIT    (CONSTANT_FILL)  // Timer enable bit in GPTMCTL
#define WTIMER_32_BIT_CFG  (CONSTANT_FILL)  // GPTMCFG value for 32-bit mode
#define                (CONSTANT_FILL)  // (unnamed macro — refer to .h file)
#define PRESCALER_VALUE    (CONSTANT_FILL)  // Prescaler for 1 ms resolution
```

Test Criteria

- DELAY_1MS(500) produces a 500 ms delay, verified with oscilloscope and LED blink
- Delay accuracy is within ±5%

> **Note: LED and Button Behavior**
> LED blink (2 Hz red), mode toggle (SW1), and LED cycling (SW2) are implemented in the ButtonLED module (ButtonLED.c / ButtonLED.h), not in wtimer. See Section 5.1 for ButtonLED details.

5.3 Module 3: UART0 Serial Communication (UART0.c / UART0.h)

Property	Value
Purpose	Send debug information and sensor data to PC terminal
Hardware	UART0 peripheral (U0Rx → PA0, U0Tx → PA1)
Baud rate	Student-chosen baud rate (e.g., 115200, 9600, or 38400; not 57600)
Format	8 data bits, 1 stop bit, no parity, FIFOs enabled
Timing	Output updates every 1 second (1 Hz)
Dependencies	tm4c123gh6pm.h, util.h (wtimer.h)

Functional Requirements

Students must implement the following functions, declared in UART0.h:

Function Signature	Description
`void UART0_Init(void)`	Initialize UART0 at student-chosen baud rate (not 57600; e.g., 115200)
`void UART0_OutCRLF(void)`	Transmit carriage return + line feed (\r\n)
`unsigned char UART0_InChar(void)`	Wait for and return one received ASCII character
`void UART0_OutChar(char data)`	Transmit a single ASCII character
`void UART0_OutString(char *pt)`	Transmit a null-terminated string
`void UART0_InString(char *bufPt, unsigned short max)`	Receive characters into buffer until Enter or max length

Test Criteria

- All characters transmit without corruption at chosen baud rate
- Terminal (Putty / Tera Term / YAT) displays text correctly
- UART0_OutString correctly terminates at the null character
- Output timing is regular (±100 ms tolerance at 1 Hz)

Note: Integer and Float Formatting

UART0.h does not declare dedicated SendInteger or SendFloat functions. Use sprintf() to format numeric values into a char buffer, then call UART0_OutString() to transmit the result.

5.4 Module 4: I2C Bus Communication (I2C1–3, student choice) (I2C.c / I2C.h)

Property	Value
Purpose	Manage all I2C communication with the three external slave devices
Hardware	I2C module (SDA → PB3, SCL → PB2)
Clock	Standard Mode, 100 kHz
Pull-ups	External 4.7 kΩ resistors required on SDA and SCL
Dependencies	tm4c123gh6pm.h, util.h

Slave Addresses

Device	I2C Address
TCS34725 RGB Color Sensor	0x29
MPU6050 IMU	0x68
16x2 LCD Backpack	0x3F (LCD_WRITE_ADDR as defined in LCD.h)

Functional Requirements

Students must implement the following functions, declared in I2C.h:

Function Signature	Description
`void I2C_Init(void)`	Initialize I2C master (I2C1–3, student choice; I2C0 not allowed) at 100 kHz. Enable system clocks, configure PB2/PB3 as I2C alternate function, enable open-drain on SDA, set TPR for 100 kHz
`uint8_t I2C_Receive(uint8_t slave_addr, uint8_t slave_reg_addr)`	Send register address, generate repeated START, read one byte. Returns received byte
`uint8_t I2C_Transmit(uint8_t slave_addr, uint8_t slave_reg_addr, uint8_t data)`	Write one byte to the specified register. Returns 0 on success, non-zero on error
`void I2C_Burst_Receive(uint8_t slave_addr, uint8_t slave_reg_addr, uint8_t* data, uint32_t size)`	Read multiple consecutive bytes starting at slave_reg_addr into data buffer
`uint8_t I2C_Burst_Transmit(uint8_t slave_addr, uint8_t slave_reg_addr, uint8_t* data, uint32_t size)`	Write multiple consecutive bytes from data buffer. Returns 0 on success

CONSTANT_FILL Macros (I2C.h)

```
// Clock gating
```

```
#define EN_I2C_CLOCK       (CONSTANT_FILL)  // SYSCTL_RCGI2C_R bit for I2C
#define EN_GPIOB_CLOCK      (CONSTANT_FILL)  // SYSCTL_RCGCGPIO_R bit for Port B

// GPIO alternate function setup
#define I2C_PINS            (CONSTANT_FILL)  // Digital enable mask for PB2 and PB3
#define I2C_ALT_FUNC_MSK    (CONSTANT_FILL)  // AFSEL mask
#define I2C_ALT_FUNC_SET    (CONSTANT_FILL)  // PCTL value for I2C
#define I2C_SDA_PIN         (CONSTANT_FILL)  // Open-drain enable bit (PB3)
#define I2C_SCL_PIN         (CONSTANT_FILL)  // SCL pin bit (PB2)

// I2C master configuration
#define EN_I2C_MASTER       (CONSTANT_FILL)  // MCR bit to enable master mode
#define I2C_MTPR_TPR_VALUE  (CONSTANT_FILL)  // TPR = (SysClk / (2*10*SCL_CLK)) - 1
#define I2C_MTPR_STD_SPEED  (CONSTANT_FILL)  // Standard-speed mode bit
#define I2C_RW_PIN         (CONSTANT_FILL)   // R/W bit position in MSA register
#define RUN_CMD             (CONSTANT_FILL)  // MCS RUN bit
```

TPR Calculation Reference

```
TPR = (System Clock / (2 * (SCL_LP + SCL_HP) * SCL_CLK)) - 1
      SCL_LP = 6,  SCL_HP = 4  (fixed by hardware)

Example: 40 MHz system clock, 100 kHz target
  TPR = (40,000,000 / (2 * 10 * 100,000)) - 1 = 19
```

Test Criteria

- I2C clock measured at exactly 100 kHz on oscilloscope
- ACK bit visible after each address byte on SDA
- All three slave devices respond correctly to I2C_Receive and I2C_Transmit
- Timeout handling prevents the system from hanging on a missing device

5.5 Module 5: TCS34725 RGB Color Sensor (TCS34725.c / TCS34725.h)

Property	Value
Purpose	Detect the dominant color of objects using raw RGB and clear data
I2C Address	0x29 (TCS34725_ADDR)
ID Register	TCS34725_ID_R_ADDR — must return TCS34725_ID (0x44)
Integration	2.4 ms (ATIME register, set during TCS34725_Init)
Gain	1× (set during TCS34725_Init)
Dependencies	I2C.h, UART0.h, util.h

Data Types (defined in TCS34725.h)

```
/* Enum returned by Detect_Color() */
typedef enum {
    RED_DETECT    = 0,
    GREEN_DETECT  = 1,
    BLUE_DETECT   = 2,
    NOTHING_DETECT = 3
} COLOR_DETECTED;

/* Struct to hold raw and normalized RGB + clear values */
typedef struct {
    uint16_t R_RAW;   // Raw 16-bit red channel
    uint16_t G_RAW;   // Raw 16-bit green channel
    uint16_t B_RAW;   // Raw 16-bit blue channel
    uint16_t C_RAW;   // Raw 16-bit clear channel (used for normalization)
    float    R;       // Normalized red   (0.0 to 255.0)
    float    G;       // Normalized green (0.0 to 255.0)
    float    B;       // Normalized blue  (0.0 to 255.0)
} RGB_COLOR_HANDLE_t;
```

Functional Requirements

Students must implement the following functions, declared in TCS34725.h:

Function Signature	Description
`void TCS34725_Init(void)`	Verify device ID; set integration time to 2.4 ms; set gain to 1×; power on sensor; enable RGBC. Prints status to UART0 at each step.
`uint16_t TCS34725_GET_RAW_CLEAR(void)`	Read and return 16-bit clear channel data (CDATAL + CDATAH)
`uint16_t TCS34725_GET_RAW_RED(void)`	Read and return 16-bit red channel data (RDATAL + RDATAH)
`uint16_t TCS34725_GET_RAW_GREEN(void)`	Read and return 16-bit green channel data (GDATAL + GDATAH)

Function Signature	Description
`uint16_t TCS34725_GET_RAW_BLUE(void)`	Read and return 16-bit blue channel data (BDATAL + BDATAH)
`void TCS34725_GET_RGB( RGB_COLOR_HANDLE_t * inst)`	Populate inst→R_RAW / G_RAW / B_RAW / C_RAW from sensor; normalize R/G/B to 0–255 by dividing by C_RAW. Guard against division by zero.
`COLOR_DETECTED Detect_Color( RGB_COLOR_HANDLE_t* inst)`	Compare normalized R, G, B values; return the dominant COLOR_DETECTED enum. Returns NOTHING_DETECT if no clear dominant color.

Key Register Macros (CONSTANT_FILL in TCS34725.h)

```
#define TCS34725_ADDR          (CONSTANT_FILL)  // 0x29
#define TCS34725_CMD           (CONSTANT_FILL)  // Command bit (bit 7 = 1, bit 5 =
0 for auto-increment)
#define TCS34725_ENABLE_R_ADDR (CONSTANT_FILL)  // Enable register
#define   TCS34725_ENABLE_PON  (CONSTANT_FILL)  // Power-on bit
#define   TCS34725_ENABLE_AEN  (CONSTANT_FILL)  // RGBC enable bit
#define TCS34725_TIMING_R_ADDR (CONSTANT_FILL)  // ATIME register
#define   TCS34725_ATIME_2_4_MS(CONSTANT_FILL)  // 2.4 ms integration time value
#define TCS34725_CTRL_R_ADDR   (CONSTANT_FILL)  // Control register (gain)
#define   TCS34725_CTRL_AGAIN_1(CONSTANT_FILL)  // 1x gain value
#define TCS34725_ID_R_ADDR     (CONSTANT_FILL)  // ID register
#define TCS34725_ID            (CONSTANT_FILL)  // Expected ID value (0x44)

// Color data register addresses
#define TCS34725_CDATAL_R_ADDR (CONSTANT_FILL)  // Clear low byte
#define TCS34725_CDATAH_R_ADDR (CONSTANT_FILL)  // Clear high byte
#define TCS34725_RDATAL_R_ADDR (CONSTANT_FILL)  // Red low byte
#define TCS34725_RDATAH_R_ADDR (CONSTANT_FILL)  // Red high byte
#define TCS34725_GDATAL_R_ADDR (CONSTANT_FILL)  // Green low byte
#define TCS34725_GDATAH_R_ADDR (CONSTANT_FILL)  // Green high byte
#define TCS34725_BDATAL_R_ADDR (CONSTANT_FILL)  // Blue low byte
#define TCS34725_BDATAH_R_ADDR (CONSTANT_FILL)  // Blue high byte
```

Test Criteria

- TCS34725_Init prints "TCS34725 has been Detected" to UART0; if not, prints error and returns
- Each GET_RAW function returns a non-zero, non-0xFFFF value under normal lighting
- TCS34725_GET_RGB populates all float fields; values change with different colored objects
- Detect_Color returns RED_DETECT, GREEN_DETECT, or BLUE_DETECT for clearly colored targets
- 3 ms DELAY_1MS calls in Init are preserved to allow integration time to elapse

5.6 Module 6: MPU6050 6-DOF IMU (MPU6050.c / MPU6050.h)

Property	Value
Purpose	Measure 3-axis acceleration and 3-axis rotation rate; compute tilt angles
I2C Address	0x68 (MPU6050_ADDR_AD0_LOW when AD0 is low)
ID Register	WHO_AM_I (0x75) — must return 0x68
Data format	16-bit signed integers in 2's complement, split across consecutive H/L register pairs
Dependencies	I2C.h, UART0.h, tm4c123gh6pm.h, math.h

Scale Factors (defined as local constants in MPU6050.c)

```
// Accelerometer LSB sensitivity (set by ACCEL_CONFIG register bits [4:3])
#define ACCEL_LSB_0_VALUE   (16384.0)  // ±2g  range  →  16,384 LSB/g
#define ACCEL_LSB_1_VALUE   ( 8192.0)  // ±4g  range  →   8,192 LSB/g
#define ACCEL_LSB_2_VALUE   ( 4096.0)  // ±8g  range  →   4,096 LSB/g
#define ACCEL_LSB_3_VALUE   ( 2048.0)  // ±16g range  →   2,048 LSB/g

// Gyroscope LSB sensitivity (set by GYRO_CONFIG register bits [4:3])
#define GYRO_LSB_0_VALUE    (131.0)    // ±250 °/s  →  131   LSB/(°/s)
#define GYRO_LSB_1_VALUE    ( 65.5)    // ±500 °/s  →   65.5 LSB/(°/s)
#define GYRO_LSB_2_VALUE    ( 32.8)    // ±1000°/s  →   32.8 LSB/(°/s)
#define GYRO_LSB_3_VALUE    ( 16.4)    // ±2000°/s  →   16.4 LSB/(°/s)

#define RAD_TO_DEGREE_CONV  (180 / 3.1415)  // Used in MPU6050_Get_Angle()
```

Data Types (defined in MPU6050.h)

```
typedef struct {
    int16_t Ax_RAW, Ay_RAW, Az_RAW;  // Raw 16-bit signed accelerometer counts
    float   Ax, Ay, Az;              // Processed values in g
} MPU6050_ACCEL_t;

typedef struct {
    int16_t Gx_RAW, Gy_RAW, Gz_RAW;  // Raw 16-bit signed gyroscope counts
    float   Gx, Gy, Gz;              // Processed values in °/s
} MPU6050_GYRO_t;

typedef struct {
    float ArX, ArY, ArZ;             // Tilt angles in degrees
} MPU6050_ANGLE_t;
```

Functional Requirements

Students must implement the following functions, declared in MPU6050.h:

Function Signature	Description
`void MPU6050_Init(void)`	Read WHO_AM_I; print ID or error to UART0; reset device; wake up sensor; set sample rate to 1 kHz; apply default config/accel/gyro settings
`void MPU6050_Get_Accel( MPU6050_ACCEL_t* Accel_Instance)`	Read all 6 raw accelerometer bytes (ACCEL_XOUT_H/L through ACCEL_ZOUT_H/L) via I2C; assemble 16-bit signed values and store in struct
`void MPU6050_Get_Gyro( MPU6050_GYRO_t* Gyro_Instance)`	Read all 6 raw gyroscope bytes (GYRO_XOUT_H/L through GYRO_ZOUT_H/L) via I2C; assemble 16-bit signed values and store in struct
`void MPU6050_Process_Accel( MPU6050_ACCEL_t * Accel_Instance)`	Read ACCEL_CONFIG register to determine active LSB sensitivity; divide RAW values by the appropriate ACCEL_LSB_x_VALUE; store result in Ax/Ay/Az
`void MPU6050_Process_Gyro( MPU6050_GYRO_t* Gyro_Instance)`	Divide RAW gyroscope values by the appropriate GYRO_LSB_x_VALUE; store result in Gx/Gy/Gz
`void MPU6050_Get_Angle( MPU6050_ACCEL_t* Accel_Instance, MPU6050_GYRO_t* Gyro_Instance, MPU6050_ANGLE_t* Angle_Instance)`	Use atan2() with processed Accel values and RAD_TO_DEGREE_CONV to compute ArX, ArY, ArZ in degrees
`uint8_t MPU6050_Read_Reg(uint8_t reg)`	Read a single register for debugging purposes (returns raw byte)

Key Register Macros (CONSTANT_FILL in MPU6050.h)

```
#define MPU6050_ADDR_AD0_LOW  (CONSTANT_FILL)  // 0x68  (AD0 pulled low)
#define SMPLRT_DIV            (CONSTANT_FILL)  // Sample rate divider register addr
#define   SMPLRT_DIV_8        (CONSTANT_FILL)  // Value for 1 kHz / 8 = 125 Hz
sample rate
#define CONFIG                (CONSTANT_FILL)  // Configuration register addr
#define   CONFIG_DFPL_0       (CONSTANT_FILL)  // Digital low-pass filter setting 0
#define GYRO_CONFIG           (CONSTANT_FILL)  // Gyroscope config register addr
#define   GYRO_FS_SEL_0       (CONSTANT_FILL)  // ±250 °/s full-scale
#define ACCEL_CONFIG          (CONSTANT_FILL)  // Accelerometer config register
addr
#define   ACCEL_AFS_SEL_0     (CONSTANT_FILL)  // ±2g full-scale
#define ACCEL_XOUT_H / _L     (CONSTANT_FILL)  // 0x3B / 0x3C
#define ACCEL_YOUT_H / _L     (CONSTANT_FILL)  // 0x3D / 0x3E
#define ACCEL_ZOUT_H / _L     (CONSTANT_FILL)  // 0x3F / 0x40
#define GYRO_XOUT_H  / _L     (CONSTANT_FILL)  // 0x43 / 0x44
#define GYRO_YOUT_H  / _L     (CONSTANT_FILL)  // 0x45 / 0x46
#define GYRO_ZOUT_H  / _L     (CONSTANT_FILL)  // 0x47 / 0x48
#define PWR_MGMT_1            (CONSTANT_FILL)  // Power management 1 register
(0x6B)
#define   PWR_CLK_SEL_INTERNAL(CONSTANT_FILL)  // Internal 8 MHz oscillator
#define   PWR_DEVICE_RESET    (CONSTANT_FILL)  // Reset bit
#define WHO_AM_I              (CONSTANT_FILL)  // ID register (0x75)
```

Test Criteria

- MPU6050_Init prints device ID to UART0; exits with error message if not 0x68
- MPU6050_Get_Accel populates all three Ax_RAW / Ay_RAW / Az_RAW fields
- Az_RAW ≈ 16,384 when board is flat and stationary (1g in ±2g mode)
- MPU6050_Process_Accel yields Az ≈ 1.0 when stationary
- All RAW values change when the board is moved or tilted
- MPU6050_Get_Angle produces plausible angle values (±90° range) as board is tilted
- Data refreshes within 100 milliseconds

5.7 Module 7: 16×2 LCD Display with I2C Backpack (LCD.c / LCD.h)

Property	Value
Purpose	Display color names and angle values; startup name sequence
I2C Address	0x3F (LCD_WRITE_ADDR as defined in LCD.h)
Backpack	PCF8574A (PCF8574A_REG = 0x00)
Display	16 characters × 2 rows, 4-bit mode
Dependencies	tm4c123gh6pm.h, wtimer.h (for DELAY_1MS), I2C.h

Important: I2C Address

The LCD backpack I2C address must match your hardware. The project PDF specifies 0x27 (PCF8574T). The starter code uses 0x3F (PCF8574A). Verify your physical backpack chip before proceeding. Verify this matches your hardware before proceeding.

Functional Requirements

Students must implement the following public and local functions:

Function Signature	Description
`static void LCD_Send_CMD(uint8_t cmd)`	Local helper: split cmd into upper and lower nibbles; construct 4-byte array with RS=0, EN pulses, and backlight bit; burst-transmit via I2C_Burst_Transmit
`static void LCD_Send_Data(uint8_t data)`	Local helper: same as LCD_Send_CMD but sets RS=1 to indicate character data
`void LCD_Init(void)`	Send INIT_REG_CMD and INIT_FUNC_CMD; configure 4-bit, 2-row, 5×7 mode; turn off display; clear; set entry mode; turn on display with cursor and blink
`void LCD_Clear(void)`	Send CLEAR_DISP_CMD; delay 1 ms for command to execute
`void LCD_Set_Cursor(uint8_t row, uint8_t col)`	Send FIRST_ROW_CMD or SECOND_ROW_CMD offset by col to position cursor
`void LCD_Reset_Cursor(void)`	Send RETURN_HOME_CMD to move cursor to row 1, column 0

Function Signature	Description
`void LCD_Print_Char(uint8_t data)`	Call LCD_Send_Data() for a single character byte
`void LCD_Print_Str(uint8_t* str)`	Iterate over null-terminated string and call LCD_Print_Char() for each byte

Key Macros (CONSTANT_FILL in LCD.h)

```
// PCF8574A control byte structure (bits sent to I2C)
#define RS_Pin          (CONSTANT_FILL)  // Register Select (bit 0: 0=cmd, 1=data)
#define RW_Pin          (CONSTANT_FILL)  // Read/Write (bit 1, always 0 for write)
#define EN_Pin          (CONSTANT_FILL)  // Enable pulse (bit 2)
#define BACKLIGHT       (CONSTANT_FILL)  // Backlight on bit (bit 3)

// LCD command codes
#define FUNC_MODE       (CONSTANT_FILL)  // Function set command base
#define   FUNC_4_BIT    (CONSTANT_FILL)  // 4-bit interface bit
#define   FUNC_2_ROW    (CONSTANT_FILL)  // 2-line display bit
#define   FUNC_5_7      (CONSTANT_FILL)  // 5x7 font bit
#define DISP_CMD        (CONSTANT_FILL)  // Display control command base
#define   DISP_ON       (CONSTANT_FILL)  // Display on bit
#define   DISP_CURSOR_ON(CONSTANT_FILL)  // Cursor visible bit
#define   DISP_BLINK_ON (CONSTANT_FILL)  // Cursor blink bit
#define CLEAR_DISP_CMD  (CONSTANT_FILL)  // Clear display command
#define ENTRY_MODE_CMD  (CONSTANT_FILL)  // Entry mode command base
#define   ENTRY_INC_CURSOR (CONSTANT_FILL) // Increment cursor on write
#define RETURN_HOME_CMD (CONSTANT_FILL)  // Return cursor to home
#define FIRST_ROW_CMD   (CONSTANT_FILL)  // DDRAM address for row 1 col 0
#define SECOND_ROW_CMD  (CONSTANT_FILL)  // DDRAM address for row 2 col 0

// Nibble extraction
#define UPPER_NIBBLE_MSK (CONSTANT_FILL) // 0xF0 mask for upper nibble
#define NIBBLE_SHIFT     (CONSTANT_FILL) // 4-bit right-shift for lower nibble
#define LCD_ROW_SIZE     (CONSTANT_FILL) // 16 characters per row
```

4-Byte Burst Transmit Sequence (LCD_Send_CMD / LCD_Send_Data)

Each nibble requires two I2C bytes to generate the Enable pulse:

```
cmd_array[0] = upper_nibble | BACKLIGHT | EN_Pin;   // Upper nibble, EN high
cmd_array[1] = upper_nibble | BACKLIGHT;            // Upper nibble, EN low
cmd_array[2] = lower_nibble | BACKLIGHT | EN_Pin;   // Lower nibble, EN high
cmd_array[3] = lower_nibble | BACKLIGHT;            // Lower nibble, EN low

// Then: I2C_Burst_Transmit(LCD_WRITE_ADDR, PCF8574A_REG, cmd_array, 4);
```

Display Content

Phase	Content
Startup (loop)	Row 1: first name → wait 1 s; Row 2: last name → wait 1 s; clear; repeat
Full system	Row 1: "Color: RED" / "Color: GREEN" / "Color: BLUE"

Phase	Content
Full system	Row 2: "Angle: 45°" (or current computed angle)
Update rate	Every 1 second (1 Hz)

Test Criteria

- Text appears on display without garbage characters
- Both rows are independently addressable and independently clear
- Backlight is on (BACKLIGHT bit set in every byte)
- Startup name sequence timing matches specification (1 s per line)
- LCD_Clear takes effect within 2 ms

5.8 Module 8: Servo Motor Control via PWM (Servo.c / Servo.h)

Property	Value
Purpose	Generate hardware PWM to position a standard RC servo motor
PWM Output	Student-chosen PWM output (M0PWM1–7 or M1PWM0–7; M0PWM0 not allowed)
Frequency	50 Hz (20 ms period)
Pulse range	1.0 ms (5% duty) to 2.0 ms (10% duty) – full ±90° range
Angle range	SERVO_MIN_ANGLE to SERVO_MAX_ANGLE (CONSTANT_FILL, typically −90 to +90)
Dependencies	tm4c123gh6pm.h, util.h (map function from wtimer)

Functional Requirements

Students must implement the following functions, declared in Servo.h:

Function Signature	Description
`void Servo_Init(void)`	Enable student-chosen PWM module and GPIO clocks; configure pin as alternate function for chosen PWM output; set PWM clock divider; configure PWM generator in count-down mode; set load (counter) value for 50 Hz; enable PWM output
`void Drive_Servo(int16_t angle)`	Map input angle (SERVO_MIN_ANGLE to SERVO_MAX_ANGLE) to a PWM compare value (SERVO_MIN_CNT to SERVO_MAX_CNT) using map(); write compare value to PWM comparator register

CONSTANT_FILL Macros (Servo.h)

```
// Clock enable
#define EN_PWM0_GPIOB_CLOCK (CONSTANT_FILL)  // SYSCTL bits for PWM0 + GPIOB
#define EN_PWM0_CLOCK       (CONSTANT_FILL)  // SYSCTL_RCGCPWM_R bit for Module 0

// GPIO alternate function
#define PWM0_PIN            (CONSTANT_FILL)  // PB6 pin bit
#define CLEAR_ALT_FUNCTION  (CONSTANT_FILL)  // PCTL mask to clear PB6 field
#define PWM0_ALT_FUNCTION   (CONSTANT_FILL)  // PCTL value for chosen PWM output
pin

// PWM clock divider
#define EN_USE_PWM_DIV      (CONSTANT_FILL)  // SYSCTL_RCC_R bit to use PWM divisor
#define CLEAR_PWM_DIV       (CONSTANT_FILL)  // Mask to clear divider field
#define PWM0_DIV_VALUE      (CONSTANT_FILL)  // Divider setting (e.g., /64)

// PWM generator configuration
#define PWM0_DEFAULT_CONFIG (CONSTANT_FILL)  // Generator 0 control register setup
#define PWM0_GEN_CONFIG     (CONSTANT_FILL)  // Generator 0 action register
```

```
#define PWM0_COUNTER        (CONSTANT_FILL)  // Load value for 50 Hz period
#define PWM0_START          (CONSTANT_FILL)  // Generator 0 enable bit
#define EN_PWM0_FUNCTION    (CONSTANT_FILL)  // PWM output enable bit

// Servo pulse width limits (as PWM compare counts)
#define SERVO_MIN_CNT       (CONSTANT_FILL)  // Compare count for 0.5 ms pulse
#define SERVO_MAX_CNT       (CONSTANT_FILL)  // Compare count for 2.5 ms pulse

// Angle limits
#define SERVO_MIN_ANGLE     (CONSTANT_FILL)  // Typically -90
#define SERVO_MAX_ANGLE     (CONSTANT_FILL)  // Typically +90
```

Angle-to-PWM Mapping

```
// Drive_Servo() uses map() from wtimer.h:
uint16_t cnt = map(angle,
                   SERVO_MIN_ANGLE, SERVO_MAX_ANGLE,
                   SERVO_MIN_CNT,   SERVO_MAX_CNT);
PWM0_0_CMPA_R = cnt;

// Reference pulse widths for a typical servo:
// -90°  →  0.5 ms pulse  (2.5% duty cycle)
//   0°  →  1.5 ms pulse  (7.5% duty cycle)
// +90° → 2.0 ms pulse (10% duty cycle)
```

Initial Motion Sequence (Unit Test)

When running Servo module in isolation, Drive_Servo() must step through:

```
Drive_Servo(0);    DELAY_1MS(1000);
Drive_Servo(-45);  DELAY_1MS(1000);
Drive_Servo(0);    DELAY_1MS(1000);
Drive_Servo(+45);  DELAY_1MS(1000);
Drive_Servo(0);    DELAY_1MS(1000);
Drive_Servo(-90);  DELAY_1MS(1000);
Drive_Servo(0);    DELAY_1MS(1000);
Drive_Servo(+90);  DELAY_1MS(1000);
Drive_Servo(0);    DELAY_1MS(1000);
```

Test Criteria

- PWM frequency measured at exactly 50 Hz on oscilloscope
- Pulse width at 0° ≈ 1.5 ms; at +90° ≈ 2.5 ms; at −90° ≈ 0.5 ms
- Servo moves smoothly through all nine positions in the motion sequence
- 1-second delay between each position is accurate (±5%)
- No jitter or instability at any static position

6 Step 2: Module Design & Implementation Plan

Students have completed Step 1 (Requirements & System Design). This section documents the Step 2 deliverables required by the 3-Step Workflow: team member assignments, a 12-day implementation schedule, module dependency map, and the traceability table. No integration is permitted until all modules pass their individual tests.

6.1 Team Member Assignments

Member	Role / Specialty	Modules Assigned	Test Due (Day)	Notes
Member A	Foundation & Timing	M1: WTIMER, M2: ButtonLED	Day 3	Critical path — all modules need DELAY_1MS()
Member B	Serial Comms & I2C Bus	M3: UART0, M4: I2C	Day 6	I2C must pass before sensor HW test
Member C	Sensors	M5: TCS34725, M6: MPU6050	Day 9	Logic prep Days 4–5; HW validate Days 7–9
Member D	Output Peripherals	M7: LCD, M8: Servo	Day 9	Servo starts Day 4; LCD HW after M4 ready

6.2 Module Dependency Map

Modules are organized in four dependency layers. No module may be integrated until the modules it depends on have passed their individual unit tests.

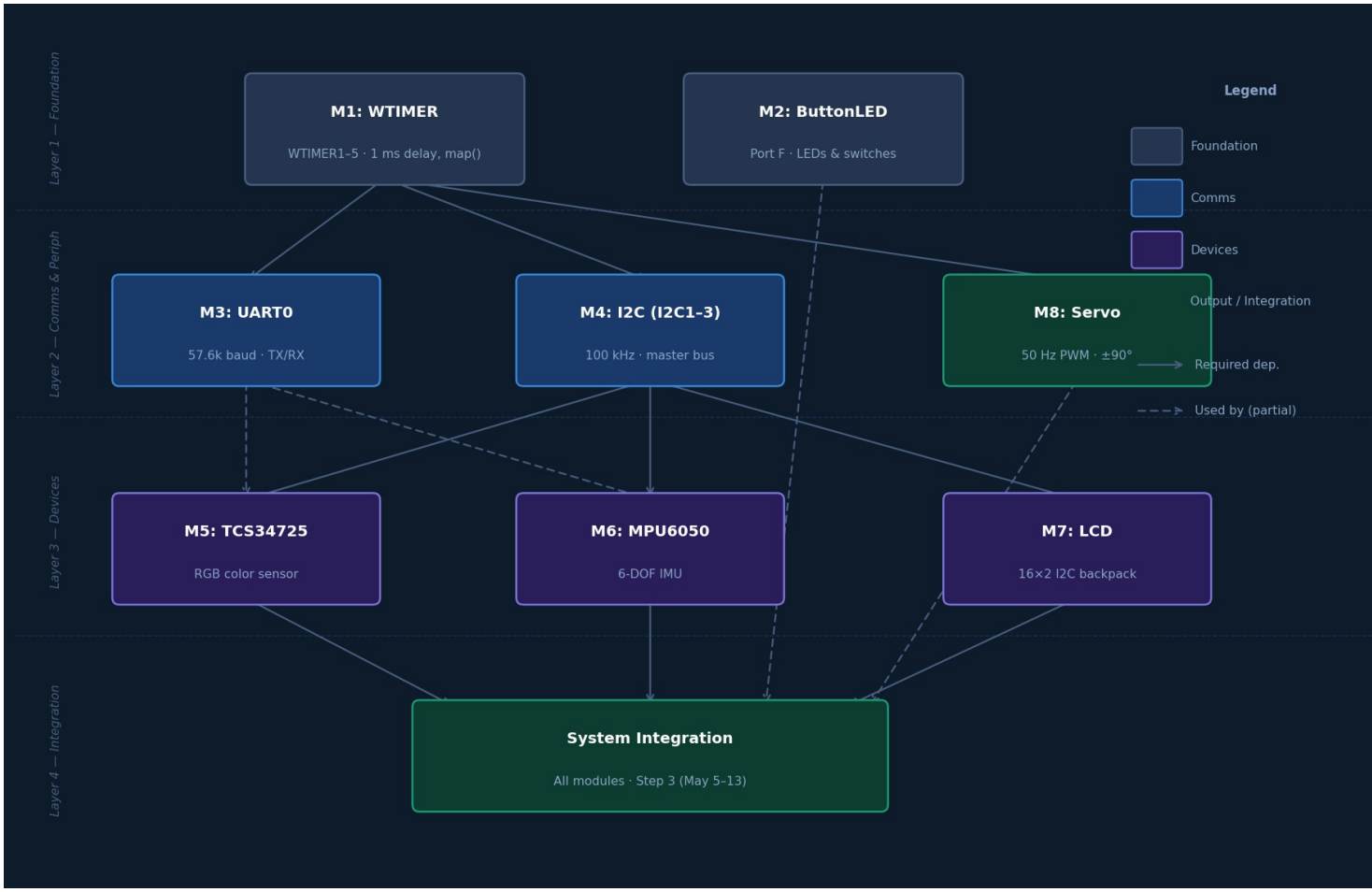

Figure 1: Module dependency graph — four layers, solid arrows = required dependency, dashed = partial use

6.3 12-Day Implementation Schedule (Step 2: April 23 – May 4)

Figure 2: 12-day implementation schedule (Step 2: April 23 – May 4) — darker bar shade = verification day

6.4 Requirement-to-Module Traceability Table

Required by the 3-Step Workflow Step 2 deliverables. Each functional requirement from Section 4 of the project description maps to the implementing module and its test criterion.

Req ID	Type	Requirement / Constraint	Spec §	Module (file : function)	Assigned	Mod TC	Int TC	Sys TC	Evidence Path	Status	CER Claim	Notes / Risks
Step 1: Reqts & System Design (Apr 20-22) | Step 2: Module Design & Implementation (Apr 23 - May 4) | Step 3: Integration, Validation & Submission (May 5-13)												
🔵 **Functional Requirements (RQ-xx) — Complete all 20 rows**												
RQ-01	Functional	WTIMER_Init() initialises wide timer (WTIMER1-5, student choice) for 1 ms delay	§5.1	*wtimer.c : WTIMER_Init*	Jackie	TC-MOD-001	TC-INT-010	TC-SYS-001	*evidence/TC-MOD-001_timer_init.png*	🔴 Not Started	Timer inits without error; 1 ms delay by oscilloscope	⭐ *EXAMPLE — WTIMER0 not allowed.*
RQ-02	Functional	DELAY_1MS(n) blocks for n ms ±5% accuracy	§5.1	*wtimer.c : DELAY_1MS*	Jackie	TC-MOD-002	TC-INT-010	TC-SYS-001	*evidence/TC-MOD-002_delay_acc.png*	🔴 Not Started	500 ms measured 475-525 ms on oscilloscope	Test at 100, 500, 1000 ms.
RQ-	Functi	LED_Init() configures	§5.	*ButtonLED.c*	Jac	TC-	TC-	TC-	*evidenc*	🔴	All 3 LED	Unlock

03	onal	PF[3:1] outputs; BTN_Init() configures PF4/PF0 with pull-ups	2	*: LED_Init, BTN_Init*	kie	MOD-003	INT-011	SYS-002	*e/TC-MOD-003_led.png*	Not Started	colours light; buttons debounce	PF4/PF0 before configuring.
RQ-04	Functional	SW1 toggles UART/WTIMER mode; SW2 cycles LED Red-Green-Blue	§5.2	*ButtonLED.c : SW1_Handler, SW2_Handler*	Jackie	TC-MOD-004	TC-INT-011	TC-SYS-002	*evidence/TC-MOD-004_mode.mp4*	🔴 Not Started	Mode toggles on each SW1 press; colour cycles on SW2	Edge-trigger on release.
RQ-05	Functional	UART0_Init() at student-chosen baud (not 57600); all TX/RX functions work	§5.3	*uart0.c : UART0_Init*	Min	TC-MOD-005	TC-INT-012	TC-SYS-003	*evidence/TC-MOD-005_uart.png*	🔴 Not Started	Terminal receives chars at chosen baud without corruption	Document chosen baud rate.
RQ-06	Functional	UART formats integers & floats via sprintf; updates at 1 Hz	§5.3	*uart0.c : SendInteger, SendFloat*	Min	TC-MOD-006	TC-INT-012	TC-SYS-003	*evidence/TC-MOD-006_fmt.png*	🔴 Not Started	Numeric output correct; 1 Hz cadence ±100 ms	Use sprintf to char buffer then SendString.
RQ-07	Functional	I2C_Init() configures I2C1-3 (student choice) at 100 kHz Standard Mode	§5.4	*i2c.c : I2C_Init*	Jackie	TC-MOD-007	TC-INT-013	TC-SYS-004	*evidence/TC-MOD-007_100k.png*	🔴 Not Started	Oscilloscope : 100 kHz + ACK after each address byte	I2C0 not allowed; 4.7 kΩ pull-ups required.
RQ-08	Functional	I2C single-byte and burst read/write communicate with all 3 slaves	§5.4	*i2c.c : I2C_Read, I2C_Write*	Jackie	TC-MOD-008	TC-INT-013	TC-SYS-004	*evidence/TC-MOD-008_slaves.png*	🔴 Not Started	All 3 devices ACK; data non-0xFF and non-0x00	Test each slave separately.
RQ-09	Functional	TCS34725_Init() verifies ID=0x44; sets ATIME=2.4 ms, gain=1x	§5.5	*tcs34725.c : TCS34725_Init*	Min	TC-MOD-009	TC-INT-014	TC-SYS-005	*evidence/TC-MOD-009_tcs.png*	🔴 Not Started	UART prints "TCS34725 has been Detected"	COMMAND bit 7 required in all register writes.
RQ-10	Functional	GET_RGB() reads raw RGBC, normalises 0-255; Detect_Color() returns dominant enum	§5.5	*tcs34725.c : GET_RGB, Detect_Color*	Min	TC-MOD-010	TC-INT-014	TC-SYS-005	*evidence/TC-MOD-010_color.mp4*	🔴 Not Started	RED/ GREEN/ BLUE returned for clearly coloured targets	Guard C_RAW =0 division; keep 3 ms delay in Init.
RQ-11	Functional	MPU6050_Init() reads WHO_AM_I=0x68; wakes sensor; sets 1 kHz rate	§5.6	*mpu6050.c : MPU6050_Init*	Jackie	TC-MOD-011	TC-INT-015	TC-SYS-006	*evidence/TC-MOD-011_mpu.png*	🔴 Not Started	UART prints 0x68; sensor responds to reads	Reset via PWR_MGMT_1 before wake-up.
RQ-12	Functional	Get_Accel/Gyro() read 16-bit signed raw; Process functions convert to g / deg/s	§5.6	*mpu6050.c : Get_Accel, Get_Gyro, Process*	Jackie	TC-MOD-012	TC-INT-015	TC-SYS-006	*evidence/TC-MOD-012_data.png*	🔴 Not Started	Az_RAW ≈ 16384 flat; Az ≈ 1.0 g; values change on tilt	Burst read 6 bytes per axis group.
RQ-13	Functional	Get_Angle() computes ArX/ArY/ArZ in degrees using atan2()	§5.6	*mpu6050.c : MPU6050_Get_Angle*	Jackie	TC-MOD-013	TC-INT-015	TC-SYS-006	*evidence/TC-MOD-013_angle.mp4*	🔴 Not Started	Angles in ±90° range; change with tilt	Include math.h; use RAD_TO_DEGREE_CONV.

RQ-14	Functional	LCD_Init() sends 4-bit init via I2C backpack; powers on without garbage	§5.7	*lcd.c : LCD_Init*	Min	TC-MOD-014	TC-INT-016	TC-SYS-007	*evidence/TC-MOD-014_init.png*	🔴 Not Started	LCD on with backlight; no garbage	Verify address matches hardware (0x27 or 0x3F).
RQ-15	Functional	Print_Str() displays text; Set_Cursor() positions; Clear() clears within 2 ms	§5.7	*lcd.c : Print_Str, Set_Cursor, Clear*	Min	TC-MOD-015	TC-INT-016	TC-SYS-007	*evidence/TC-MOD-015_text.png*	🔴 Not Started	Both rows correct; cursor positions; screen clears	Startup: first name R1, last name R2, 1 s each.
RQ-16	Functional	PWM_Init() at 50 Hz on student-chosen M0PWM1-7 or M1PWM0-7 output	§5.8	*pwm.c : PWM_Init*	Jackie	TC-MOD-016	TC-INT-017	TC-SYS-008	*evidence/TC-MOD-016_50hz.png*	🔴 Not Started	Oscilloscope: 50 Hz confirmed	M0PWM0 not allowed.
RQ-17	Functional	Drive_Servo(angle) maps ±90° to 1.0-2.0 ms pulse; 9-position sequence completes	§5.8	*pwm.c : Drive_Servo*	Jackie	TC-MOD-017	TC-INT-017	TC-SYS-008	*evidence/TC-MOD-017_seq.mp4*	🔴 Not Started	0°=1.5 ms, +90°=2.0 ms, -90°=1.0 ms; all 9 positions	Use map() for linear interpolation.
RQ-18	Functional	System samples color and angle every 100 ms in main loop	§6.2	*app.c : Main_Loop*	Both	TC-MOD-018	TC-INT-020	TC-SYS-009	*evidence/TC-MOD-018_loop.png*	🔴 Not Started	100 ms period ±5% confirmed	WTIMER interrupt preferred over busy-wait.
RQ-19	Functional	LCD and UART update every 1 second (1 Hz ±100 ms)	§6.2	*app.c : Output_Update*	Both	TC-MOD-019	TC-INT-021	TC-SYS-009	*evidence/TC-MOD-019_1hz.png*	🔴 Not Started	UART timestamps confirm 1 s cadence	Separate 1 s counter from 100 ms counter.
RQ-20	Functional	Servo follows IMU tilt in real-time; smooth movement, no jumps	§6.2	*app.c : Servo_Control*	Both	TC-MOD-020	TC-INT-022	TC-SYS-010	*evidence/TC-MOD-020_rt.mp4*	🔴 Not Started	Servo tracks tilt continuously; no jitter	Clip to ±90° before Drive_Servo().
🟡 **Constraint Requirements (CN-xx)**												
CN-01	Constraint	Sensor sampling every 100 ms; LCD & UART output every 1 s (±100 ms)	§6.2	*app.c : Main_Loop*	Both	TC-MOD-021	TC-INT-024	TC-SYS-008	*evidence/TC-MOD-021_timing.png*	🔴 Not Started	Loop timer confirms 100 ms sample and 1 s output	⭐ *EXAMPLE — Use WTIMER interrupt, not busy-wait.*
CN-02	Constraint	I2C1-3 only (not I2C0); WTIMER1-5 only (not WTIMER0); M0PWM1-7 or M1PWM0-7 (not M0PWM0); UART baud not 57600	§4.1	*wtimer.c, i2c.c, pwm.c, uart0.c : Init*	Both	CR-001			*evidence/CR-001_periph_choice.pdf*	🔴 Not Started	All four constraints verified against spec §4.1	Document chosen instance numbers.
CN-03	Constraint	External 4.7 kΩ pull-up resistors on I2C SDA and SCL	§5.4	*Hardware schematic*	Both	CR-002			*evidence/CR-002_pullup.jpg*	🔴 Not Started	I2C waveform: clean edges within Standard Mode rise-time	Without pull-ups I2C will not work reliably.
🟣 **Design Requirements (DES-xx)**												

DES-01	Design	Multiple I2C designs considered; single-bus shared SDA/SCL with pull-ups chosen	§2.1	*i2c.c; schematic*	Both				*evidence/ DES-001_design.pdf*	🔴 Not Started	Shared bus minimises GPIO; pull-ups meet rise-time spec	⭐ *EXAMPLE — Document vs software I2C trade-offs.*
DES-02	Design	Student-chosen WTIMER/I2C/PWM instances documented with pin and register rationale	§4.1	*All .c : Init*	Both				*evidence/ DES-002_rationale.pdf*	🔴 Not Started	Chosen instances avoid pin conflicts; verified in datasheet	Document chosen numbers in report.
🔴 **System Requirements (SYS-xx)**												
SYS-01	System	All 7 modules init, all 3 I2C devices detected, main loop starts without hang	§6.2	*app.c : main()*	Both			TC-SYS-009	*evidence/TC-SYS-009_init.mp4*	🔴 Not Started	UART shows all OK; loop starts within 3 s of power-on	⭐ *EXAMPLE — Init failure prints diagnostic, not hang.*
SYS-02	System	LCD shows color and angle; UART shows formatted data; servo tracks IMU in real-time	§6.3	*app.c : Output_Update, Servo_Control*	Both			TC-SYS-010	*evidence/TC-SYS-010_demo.mp4*	🔴 Not Started	LCD R1=Color:X, R2=Angle:X; servo moves with tilt	Demo video shows all outputs simultaneously.

Update Status as each test passes: 🔴 Not Started → 🟡 In Progress → 🟢 Done. Save all evidence files and submit with Step 2 deliverables. Integration (Step 3) must not begin until all RQ rows show 🟢 Done.